

Sesiunea 3: Testarea Statică (Static Testing)

Capitolul 1: Ce este Testarea Statică și de ce căutăm defecte înainte de scrierea codului?	2
1. Definiție: Testarea Statică vs. Testarea Dinamică	2
2. Ce putem testa static? (Artefactele)	2
3. De ce este Testarea Statică o „Mină de Aur”? (Beneficiile Economice).....	2
💡 Analogia QualiAdept: „Schița Arhitectului”	3
🧐 Exercițiu de Gândire: „Ochiul Static”	3
Capitolul 2: Tehnici și Tipuri de Review-uri (De la discuții la inspecții formale)	4
1. Cele 4 Tipuri de Review-uri (Standardul ISTQB).....	5
A. Review-ul Informal (Informal Review)	5
B. Walkthrough (Parcurgerea).....	5
C. Review-ul Tehnic (Technical Review).....	5
D. Inspecția (Inspection)	5
2. Rolurile într-o Inspecție Formală	6
🔴 Analogia QualiAdept: „Lansarea unei Cărți”	6
💡 Știați că...?	7
🧐 Exercițiu de Gândire: „Alege Instrumentul Potrivit”	7
Capitolul 3: Recapitulare, Concluzii și Studiu de Caz (QA Review în Acțiune)	7
1. Recapitulare Exhaustivă: Ce am învățat în Sesiunea 3?.....	8
2. Studiu de Caz: Cum facem un QA Review în viața reală?	8
A. Cerința Inițială (Așa cum a scris-o PO-ul)	8
B. Intervenția Testerului (QA Review-ul Static)	8
C. Cerința Rafinată (După QA Review)	9
3. Concluziile QualiAdept: „Legile Testării Statice”	9
🚀 Ce urmează?	10
Capitol Bonus: Testarea Statică Automată (Când roboții fac Code Review)	10
1. Ce sunt Analizările Statice de Cod (Static Analyzers)?	11
2. Ce tipuri de defecte găsesc acești „roboți”?	11

3. Rolul QA-ului în Analiza Statică Automată (Quality Gate)	11
💡 Analogia QualiAdept: „Corectorul Ortografic (Spellchecker-ul)”	11
🚀 Spre Modulul 2.....	12

Capitolul 1: Ce este Testarea Statică și de ce căutăm defecte înainte de scrierea codului?

🧠 Până acum am discutat despre rularea testelor, executarea pașilor și verificarea aplicațiilor în funcțiune. Aceasta este **Testarea Dinamică**. Totuși, un QA de nivel senior știe că cele mai periculoase și scumpe bug-uri pot fi eliminate fără ca măcar o singură linie de cod să fie executată. Aceasta este lumea **Testării Statice**.

1. Definiție: Testarea Statică vs. Testarea Dinamică

- **Testarea Statică (Static Testing):** Evaluarea unui artefact (document de cerințe, schemă de design, cod sursă) **fără a executa** software-ul. Se bazează pe analiză manuală (review-uri) sau analizoare automate de cod.
- **Testarea Dinamică (Dynamic Testing):** Verificarea software-ului în timp ce acesta rulează (pe un mediu de testare, cu date de test introduse).

2. Ce putem testa static? (Artefactele)

În faza statică, „obiectul de test” nu este aplicația funcțională, ci documentația și reprezentările ei:

- **User Stories și Cerințe de Business (SRS):** Depistarea inconsistențelor, ambiguităților și datelor lipsă.
- **Mockup-uri / Prototipuri de Design (Figma/Adobe XD):** Verificarea fluxurilor vizuale și a elementelor de interfață.
- **Arhitectura și Diagramele de Sistem:** Verificarea modului în care modulele sunt planificate să comunice.
- **Codul Sursă (Code Review):** Verificarea respectării standardelor de programare și a logicii interne înainte de compilare.

3. De ce este Testarea Statică o „Mină de Aur”? (Beneficiile

Economice)

Așa cum am văzut în curba costului defectelor, cu cât găsim un bug mai devreme, cu atât repararea lui este mai ieftină:

- **Eficiență maximă:** Găsirea unei greșeli într-o User Story înseamnă doar modificarea unui text în Jira. Înseamnă 0 ore irosite de dezvoltator și 0 ore irosite de QA în testare dinamică.
- **Prevenirea defectelor de producție:** Peste 50% dintre bug-urile din producție își au originea în cerințe prost înțelese sau incomplete.
- **Claritate pentru întreaga echipă:** Când QA-ul analizează cerințele static, îi obligă pe Product Owner și pe developeri să alinieze viziunea asupra produsului.



Analogia QualiAdept: „Schița Arhitectului”



Imaginează-ți că vrei să construiești o casă.

- **Testarea Statică:** Verifici planul casei desenat pe hârtie de arhitect. Observi că arhitectul a uitat să pună o ușă la baie sau că a desenat o scară care se oprește într-un zid. Ștergi linia cu guma și o redesenezi corect. Cost: **5 secunde și 0 RON**.
- **Testarea Dinamică:** Construiești casa completă, pui cărămizile, o zugrăvești și abia când te muți (în faza de execuție) încerci să intri în baie și realizezi că nu există ușă. Trebuie să spargi zidul, să refaci structura și să zugrăvești din nou. Cost: **Mii de euro și săptămâni de muncă**.



Exercițiu de Gândire: „Ochiul Static”



Analizează static următoarea cerință scrisă de un Product Owner:

„Sistemul va procesa plata utilizatorului rapid și va trimite o confirmare pe email dacă totul este OK.”

- Ce este vag / ambiguu în această cerință?
- Ce caz de eroare lipsește complet?
- Cum ai reformula această cerință pentru a fi pregătită de dezvoltare?

Capitolul 2: Tehnici și Tipuri de Review-uri (De la discuții la inspecții formale)

🎯 După ce am înțeles ce este testarea statică și de ce este vitală (economisirea bugetului), trebuie să învățăm *cum* se face. În lumea QA, analizarea unui document nu înseamnă doar să îl citești pe diagonală. Conform standardului ISTQB, există patru tipuri principale de review-uri, clasificate de la cel mai relaxat (informal) la cel mai strict (inspecția).

1. Cele 4 Tipuri de Review-uri (Standardul ISTQB)

Alegerea tipului de review depinde de maturitatea procesului din companie, de timpul disponibil și de criticitatea software-ului.

A. Review-ul Informal (Informal Review)

- **Cum funcționează:** Nu există un proces documentat. Pur și simplu îi ceri unui coleg: „Hei, poți să arunci un ochi pe Test Case-ul ăsta să vezi dacă are sens?”.
- **Cine îl conduce:** Nimeni în mod special. De obicei se face în format de *Pair Programming* sau *Pair Testing*.
- **Scop:** Găsirea rapidă și ieftină a defectelor evidente, fără a pierde timp cu birocrăția.

B. Walkthrough (Parcurgerea)

- **Cum funcționează:** Autorul documentului organizează o întâlnire și le prezintă documentul (sau codul) celorlalți, pas cu pas.
- **Cine îl conduce:** Autorul (ex: Product Owner-ul prezintă noua cerință de business echipei de programatori și QA).
- **Scop:** Crearea unei înțelegeri comune, obținerea de feedback și educarea echipei cu privire la noul modul. Este un review util pentru a alinia echipa (knowledge sharing).

C. Review-ul Tehnic (Technical Review)

- **Cum funcționează:** Un proces documentat, susținut de experți tehnici (fără management, pentru a nu exista presiune). Se folosesc liste de verificare (Checklists).
- **Cine îl conduce:** Un Moderator instruit sau un expert tehnic (NU autorul).
- **Scop:** Se asigură că documentul sau codul respectă standardele tehnice ale companiei (ex: Arhitectura de cod este corectă? S-a respectat standardul de securitate?).

D. Inspecția (Inspection)

- **Cum funcționează:** Cel mai formal, rigid și documentat tip de review. Există criterii stricte de intrare și de ieșire. Membrii citesc documentul înainte de întâlnire și vin cu defectele deja notate. Se colectează metrice (ex: câte defecte am găsit pe pagină).
- **Cine îl conduce:** Un **Moderator** dedicat și instruit. Autorul nu are voie să conducă inspecția.
- **Scop:** Găsirea defectelor critice în documentele de maximă importanță (ex: algoritmi financiari, sisteme medicale).

2. Rolurile într-o Inspecție Formală

Pentru a trece examenul ISTQB (și pentru a organiza o ședință de QA eficientă), trebuie să știi cine se așează la masa unei inspecții:

- **Autorul:** Cel care a scris documentul/codul. Rolul lui este de a răspunde la întrebări și de a repara defectele găsite.
- **Moderatorul:** Cel care conduce ședința, se asigură că discuțiile nu deviază (time management) și mediază conflictele.
- **Reviewer (Inspectorul):** Aici intervine **Testerul (QA)**. Rolul tău este să vii pregătit și să identifici defectele pe baza experienței tale.
- **Scribe (Scribul/Secretarul):** Notează toate defectele și deciziile luate pe parcursul întâlnirii.
- **Managerul:** De obicei, *NU* participă la inspecție, pentru ca autorul să nu se simtă evaluat sau amenințat.

🎯 Analogia QualiAdept: „Lansarea unei Cărți”

💡 Imaginează-ți că scrii un roman (Codul/Aplicația):

- **Review Informal:** Îi dai primul capitol soției/soțului să îl citească repede și să-ți spună dacă îi place acțiunea.
- **Walkthrough:** Îi aduni pe prietenii tăi în sufragerie și le citești tu cartea cu voce tare. Pe măsură ce citești, ei zic: „Stai, personajul ăsta nu murise în capitolul 2?”. Strângeți idei împreună.
- **Technical Review:** Trimiți manuscrisul altui scriitor profesionist. El nu judecă povestea, ci verifică dacă ai respectat regulile gramaticale și tehnicile de narațiune.
- **Inspecția Formală:** Trimiți cartea la o Casă de Editură. Acolo, un Editor Șef (Moderator) adună un Corector (Tester) și un Critic. Ei aplică o listă strictă de verificare (Checklist). Tu doar stai și asculți ce trebuie să reparați înainte de tipărire.

💡 Știați că...?

💡 În echipele IT de succes există un concept numit **„Egoless Programming”** (Programare fără ego)? Acesta spune că defectele găsite într-un review aparțin *produsului*, nu *autorului*. Când faci un review static, nu spui niciodată: „Ai greșit aici”, ci „Există o inconsecvență în acest paragraf”. Scopul review-ului este să atace problema, nu persoana!

🧑‍💻 Exercițiu de Gândire: „Alege Instrumentul Potrivit”

📝 Imaginează-ți că ești QA Lead la o companie. Ce tip de review (Informal, Walkthrough, Technical sau Inspecție) ai organiza pentru următoarele situații?

- Un developer junior tocmai a scris 10 linii de cod pentru a schimba culoarea unui buton și vrea să te asiguri că a ales nuanța corectă.
- Product Owner-ul a terminat de scris documentul cu cerințele generale pentru o nouă aplicație și vrea ca toată echipa să îl înțeleagă.
- Echipa a proiectat logica de procesare a plăților cu cardul pentru un magazin online cu milioane de utilizatori (cel mai critic modul al companiei).

Capitolul 3: Recapitulare, Concluzii și Studiu de Caz (QA Review în Acțiune)

🎯 Am ajuns la finalul Sesiunii 3. Dacă până acum am învățat teoria din spatele testării statice și procesele prin care o desfășurăm (review-urile), acum este momentul să punem teoria în practică. Un QA de succes nu doar citește standarde, ci intervine activ pentru a

„curăța” documentația proiectului.

1. Recapitulare Exhaustivă: Ce am învățat în Sesiunea 3?

- **Mindset-ul Preventiv:** Am înțeles că testarea începe *înainte* de scrierea codului. Testarea statică este despre prevenție, în timp ce testarea dinamică este despre vindecare.
- **Economia Calității (Shift Left):** Găsirea unui defect într-un document costă de 10-100 de ori mai puțin decât găsirea lui în producție, deoarece nu implică ore de programare, compilare și re-testare.
- **Spectrul Review-urilor:** Am învățat să adaptăm rigoarea verificării la importanța documentului:
 - *Informal:* Rapid, colegial, ieftin.
 - *Walkthrough:* Autorul explică, echipa învață și comentează.
 - *Technical Review:* Experții verifică standardele tehnice.
 - *Inspecția Formală:* Proces rigid, condus de un Moderator, cu metrici stricte (pentru sisteme critice).
- **Egoless Programming:** Am asimilat principiul conform căruia criticăm codul/documentul, niciodată persoana care l-a scris.

2. Studiu de Caz: Cum facem un QA Review în viața reală?

Să simulăm o ședință de rafinare (Grooming / Walkthrough) într-o echipă Agile. Product Owner-ul (PO) aduce în fața echipei o nouă funcționalitate pentru un magazin online.

A. Cerința Inițială (Așa cum a scris-o PO-ul)

- **Titlu (User Story):** Aplicare cod de reducere.
- **Descriere:** „Ca utilizator, vreau să pot introduce un cod promoțional în coșul de cumpărături pentru a plăti mai puțin.”
- **Criterii de Acceptanță (Acceptance Criteria):**
 - Utilizatorul introduce codul „REDUCERE20”.
 - Sistemul scade 20% din totalul comenzii.
 - Utilizatorul poate finaliza plata.

B. Intervenția Testerului (QA Review-ul Static)

Dacă programatorul ia această cerință și scrie codul direct, aplicația va fi plină de vulnerabilități. Ca tester QualiAdept, tu ridici mâna și adresezi următoarele întrebări (identificând defectele statice):

- **Defect de Incompletitudine (Edge Cases):**
 - QA *întreabă*: „Ce se întâmplă dacă utilizatorul introduce codul de două ori? Reducerea devine 40%?”
 - QA *întreabă*: „Ce se întâmplă dacă utilizatorul introduce 'reducere20' (cu litere mici)? Este Case Sensitive?”
- **Defect de Logică de Business:**
 - QA *întreabă*: „Reducerea de 20% se aplică doar la produse sau și la taxa de transport? (Dacă se aplică și la transport, compania pierde bani).”
- **Defect privind Lipsa Scenariilor Negative:**
 - QA *întreabă*: „Ce mesaj de eroare afișăm dacă codul a expirat sau a fost introdus greșit?”

C. Cerința Rafinată (După QA Review)

Datorită intervenției tale, Product Owner-ul modifică documentul. Noua formă arată astfel:

- **Criterii de Acceptanță Revizuite:**
 - Reducerea de 20% se aplică **doar la subtotalul produselor**, excluzând taxa de transport.
 - Codul **nu este** Case Sensitive (acceptă și „reducere20” și „REDUCERE20”).
 - Sistemul permite aplicarea unui **singur cod** per comandă. Dacă se introduce un al doilea cod, primul este suprascris, iar utilizatorul este notificat.
 - Dacă codul este invalid/expirat, apare mesajul cu text roșu: „Codul introdus nu este valid”.

Concluzia Studiului de Caz: Prin 5 minute de discuții și testare statică, ai salvat zile întregi de muncă. Ai prevenit bug-uri care ar fi permis clienților să obțină produse gratuit (prin cumularea codurilor) și ai asigurat o experiență de utilizare clară. Acesta este valoarea adăugată a unui QA Senior!

3. Concluziile QualiAdept: „Legile Testării Statice”

- **O cerință vagă este un bug în așteptare.** Niciodată nu lăsa o ambiguitate să treacă în faza de dezvoltare.

- **Review-ul atacă problema, nu omul.** Folosește un ton constructiv. Scopul este calitatea produsului, nu demonstrarea superiorității.
- **Privește dincolo de Happy Path.** Autorii documentelor descriu, de obicei, cum merg lucrurile când totul este perfect. Jobul tău este să întrebi: „*Dar dacă...?*”

Ce urmează?

💡 Cu această sesiune încheiem **Modulul 1: Fundamentele și Mindset-ul de Tester**. Acum ai gândirea, vocabularul (ISTQB) și strategia necesară. În **Modulul 2**, trecem la unelte de nivel enterprise! Vom intra în **Jira** (cel mai folosit soft de management din lume) și vom învăța cum să ne scriem Test Case-urile și să raportăm bug-urile folosind extensii dedicate precum **Zephyr** sau **TestRail**. Pregătește-te pentru multă acțiune în platformă!

Capitol Bonus: Testarea Statică Automată (Când roboții fac Code Review)

🧠 Până acum, am privit testarea statică din perspectiva *umană*: întâlniri, discuții pe documente, inspecții formale. Aceasta este jumătate din poveste. În companiile moderne de IT, Testarea Statică se face și **automatizat**, folosind unelte software care „citesc” codul scris de programatori fără să îl ruleze vreodată. Ca QA Manual, s-ar putea să nu configurezi tu aceste unelte, dar trebuie să știi să le citești

rapoartele pentru a ști cât de „riscant” este codul pe care urmează să îl testezi.

1. Ce sunt Analizoarele Statice de Cod (Static Analyzers)?

Sunt programe speciale (ex: **SonarQube**, **ESLint**, **Checkstyle**) care scanează codul sursă al aplicației imediat ce programatorul apasă pe butonul de „Save” sau încearcă să trimită codul către server. Ele caută tipare greșite, reguli încălcate sau vulnerabilități cunoscute.

2. Ce tipuri de defecte găsesc acești „roboți”?

Analiza statică automată este absolut genială la găsirea problemelor tehnice ascunse, pe care ochiul uman le-ar rata ușor:

- **Vulnerabilități de Securitate:** Găsesc parole scrise direct în cod (hardcoded passwords) sau conexiuni nesecurizate către baze de date.
- **Cod Mort (Dead Code):** Identifică variabile sau funcții care au fost scrise, dar nu sunt folosite absolut niciodată în program (ocupând memorie degeaba).
- **Încălcarea Standardelor (Coding Standards):** Verifică dacă programatorul a respectat regulile companiei (ex: dacă a pus parantezele unde trebuie sau dacă a denumit variabilele corect).
- **Complexitatea Ciclomatică:** Este o metrică ce numără câte decizii (if/else) există într-o singură funcție. Dacă o funcție are prea multe ramificații, analizorul static avertizează: *„Atenție, această bucată de cod este prea complexă și va fi un coșmar de testat și de întreținut!”*.

3. Rolul QA-ului în Analiza Statică Automată (Quality Gate)

Într-un proiect avansat, QA-ul (sau Test Managerul) colaborează cu echipa de DevOps pentru a seta un **Quality Gate (Poartă de Calitate)**.

Aceasta este o regulă strictă: dacă SonarQube (analizorul static) spune că noul cod are un scor de securitate sub nota B, sau are mai mult de 5 bug-uri tehnice, **codul este respins automat**. Nu ajunge niciodată pe mediul de testare, iar tu, ca tester, nu îți pierzi timpul cu un cod scris prost.

💡 **Analogia QualiAdept: „Corectorul Ortografic (Spellchecker-**

ul)”

Gândește-te la momentul când scrii un document în Microsoft Word.

- **Testarea Statică Manuală** înseamnă să printezi documentul și să i-l dai unui coleg să îl citească și să-ți spună dacă ideile tale au logică (Walkthrough/Inspecție).
- **Testarea Statică Automată** este linia roșie ondulată care apare **instantaneu** sub un cuvânt atunci când ai mâncat o literă (ex: „*progrmare*”). Word-ul nu a „rulat” cartea ta, nu i-a înțeles povestea, dar a scanat textul folosind un dicționar predefinit și a găsit o eroare de sintaxă în milisecunde.

Concluzia: Roboții (analizările statice) sunt perfecți pentru a găsi defecte de sintaxă și reguli tehnice încălcate (linia roșie din Word), dar doar un QA uman (Testarea Statică Manuală) poate citi un document de cerințe pentru a-ți spune că „povestea” aplicației nu are sens!

Spre Modulul 2...

Acum, având „centura neagră” în prevenție și testare statică, ești cu adevărat pregătit să intri în **Modulul 2**, unde vom învăța cum să ne documentăm munca și să raportăm bug-urile (chiar și pe cele găsite static) folosind **Jira**.